Declared vs. Observed

Why a control needs two independent witnesses, and what FedRAMP 20x gets right about evidence

Chris Morrissey · October 2026 · Whitepaper

> ⚠️ **Fictional organization. Created for portfolio demonstration purposes.** Salgado Public Systems, Inc. and the CaseBoard platform do not exist. Every metric, finding, and result cited here is invented. No certification or assessment occurred. See the disclaimer and Scope and accuracy.

Summary

A security control can be correctly configured and still not be true.

Most compliance evidence answers one question: what does the system say it does? A policy document, a configuration export, and a screenshot of a settings page are three versions of that answer. None of them shows what the system did.

Three claims follow, and the rest of this paper supports them with a worked example:

1. **Declared state and observed behavior are different kinds of evidence.** A control is demonstrated when both agree.
2. **A second validation method earns its place by being independent of the first.** Two checks that read the same source are one assertion counted twice.
3. **When the two disagree, the observed behavior is the finding.** That disagreement is the most useful thing a validation program produces.

The example is a validation engine that evaluates 13 Key Security Indicators with 26 methods for a fictional FedRAMP 20x Class C provider. Eleven indicators pass. Two fail, and both would have passed a configuration review.

The problem with declared evidence

Traditional assessment leans on narrative. A provider writes down how a control is implemented, an assessor samples artifacts that support the description, and the

result stands until the next assessment. The process is slow, and the slowness gets most of the criticism. The larger weakness is that nearly all of the evidence comes from one place: the system's own description of itself.

Configuration is a statement of intent. It records what an administrator asked for. It leaves four questions open:

- Is every resource actually governed by that configuration?
- Does something older, built before the configuration existed, sit outside it?
- Was an exception added by hand and never removed?
- Does the enforcement point behave the way the setting implies?

Each of these gaps is ordinary, and none needs negligence to appear. Any system more than a year old has some of them.

What FedRAMP 20x changes

FedRAMP's Consolidated Rules for 2026 require providers to validate **Key Security Indicators (KSIs)**, short statements of a security outcome, with automated methods. A generated Security Decision Record takes the place of the hand-written System Security Plan, and an Ongoing Certification Report takes the place of the monthly continuous-monitoring submission.

One rule carries this paper. For a Class C provider, rule `FRC-CSX-VVK` requires **at least two automated validation methods for each KSI**.

Read as a count, the rule is satisfied by writing two checks. Read for its purpose, it asks something harder: what are two methods for?

What makes a second method worth having

Two methods exist to corroborate each other, and corroboration means something only if the witnesses could disagree.

If one method reads the identity provider's policy and a second reads the same policy through a different API call, the second adds nothing. Both will be right together and wrong together. The count is two and the evidence is one.

The rule sets a count. It says nothing about where the two methods get their data. The pairing below is my design choice for meeting it, and other designs could satisfy the same rule. The engine in Project 3 pairs every KSI across a deliberate boundary:

	Control-plane method	Data-plane or telemetry method
Asks	What does the configuration declare?	What did the running system do?
Source	Policy, inventory, settings	Event logs, flow logs, active tests, job output
Fails when	The intent is wrong or missing	The intent is not enforced
Blind to	Anything outside the managed configuration	Anything that did not occur in the window

The two columns have different blind spots, and that difference is what the second method buys. Five of the pairings:

KSI	Declared	Observed
Passwordless authentication (`KSI-IAM-APM`)	IdP policy requires phishing-resistant MFA, no password fallback	48,213 authentications over 30 days, all with phishing-resistant MFA
Just-in-time privilege (`KSI-IAM-JIT`)	Zero standing privileged roles; 60-minute session cap	287 elevations, none bypassing the workflow; 2 reached the cap and were revoked at it
Change logging (`KSI-CMT-LMC`)	Logging enabled, immutable storage	1,926 change events: 1,911 reconciled to tickets, 15 to standard changes, none unreconciled
Asset inventory (`KSI-PIY-GIV`)	143 resources in the system of record	Cloud asset APIs found 147; the 4 extras sit outside the boundary with a decision on file
Security training (`KSI-CED-RAT`)	88 of 91 staff complete; the other 3 were hired in the last 30 days	Phishing simulation click rate of 2.1% against a 5% target

The last two rows show the idea reaching past infrastructure. An inventory declares what exists, and discovery observes what is running. A completion record declares that training happened, and a simulation result observes whether behavior changed.

Worked example: two indicators that pass on paper

The engine's output for CaseBoard:

```
KSI             Status   Methods  Drift
--------------------------------------------------
KSI-IAM-AAM     TRUE     2        -
KSI-IAM-APM     TRUE     2        -
KSI-IAM-ELP     TRUE     2        -
KSI-IAM-JIT     TRUE     2        -
KSI-IAM-SNU     TRUE     2        -
KSI-IAM-SUS     TRUE     2        -
KSI-CMT-LMC     TRUE     2        -
KSI-CMT-RMV     TRUE     2        -
KSI-CNA-RNT     FALSE    2        DRIFT
KSI-SVC-SIN     FALSE    2        DRIFT
KSI-CED-RAT     TRUE     2        -
KSI-PIY-GIV     TRUE     2        -
KSI-INR-RIR     TRUE     2        -
--------------------------------------------------
11/13 KSIs true | 26 validation methods
```

Finding 1: Restricting network traffic (`KSI-CNA-RNT`)

- **Declared:** default-deny on ingress and egress, seven segments, explicit policy on 100% of workloads. *Pass.*
- **Observed:** 1,317 permitted flows over 30 days that matched no explicit allow rule, all on one path: `legacy-reporting-subnet -> analytics-egress`. *Fail.*

Finding 2: Securing information (`KSI-SVC-SIN`)

- **Declared:** all 19 data stores encrypted with customer-managed keys, no keys past rotation age, TLS 1.3 minimum, FIPS-validated modules. *Pass.*
- **Observed:** active protocol negotiation across 41 endpoints found one listener, `lb-legacy-reporting-01:443`, still accepting TLS 1.0. *Fail.*

In both cases the configuration is correct and the system does not behave accordingly. A configuration review would have reported 13 of 13.

What the test suite shows

A small regression suite runs the engine three ways against the same fixtures:

Run	Evidence used	Result
A	Control-plane methods only	13 of 13 true. Both drift conditions are missed.

Run	Evidence used	Result
B	The network policy read run twice under two labels	The KSI reports true and meets the two-method count. The drift is missed. The engine marks it as not independently corroborated.
C	Control-plane plus data-plane or telemetry	11 of 13 true. Both drift conditions are detected.

Run B is the case the method count cannot catch. Two methods satisfy the rule and see nothing, because they are the same evidence. An earlier version of the engine judged independence by method label and marked this pair as corroborated. It now records which fixtures each method read while it ran and compares those. Further cases confirm that the result follows the fixtures in both directions. Clearing the two drift values gives 13 of 13, planting a single JIT bypass fails `KSI-IAM-JIT`, and one unexplained flow is enough to fail `KSI-CNA-RNT`.

These are tests against fixtures I wrote. They show that the implementation behaves as described under defined conditions. They are not a measurement of detection rates, and they say nothing about how often drift occurs in real systems.

What the results teach

1. The test is zero unexplained, and a percentage hides it

The unexpected traffic is 1,317 flows out of about 18.4 million permitted, roughly 0.007%. The TLS failure is one endpoint of 41. As coverage percentages, both controls look excellent: 99.99% and 97.6%.

The passing indicators are not clean either. Two of 417 accounts sit outside the automated lifecycle. Three of 238 deployments had no pre-merge peer review. Three of 91 staff have not completed awareness training. Four discovered resources are missing from the inventory. Secret scanning raised four findings and one was real.

What separates a pass from a fail here is whether every exception is accounted for, and the pass logic of each method says what counts:

Exception	Why the indicator still passes
2 accounts outside automated lifecycle	Sealed break-glass accounts under a documented exception

Exception	Why the indicator still passes
3 deployments without pre-merge review	Emergency changes, each reviewed afterward
3 staff without training	Hired inside the 30-day onboarding window
4 resources not in the inventory	Outside the authorization boundary, each with a boundary decision on file
1 open excess-privilege finding	Two days old, inside a five-day remediation window

The 1,317 flows fail the same test these pass. No allow rule, exception record, or boundary decision explains them. Each assertion in the engine is an absolute over the unexplained remainder (`unexcepted_manual == 0`, `unmatched == 0`, `unexpected_permitted_flows == 0`), and that is a better pass criterion than either "zero exceptions" or "above 99%". The first is false for any running system. The second lets a boundary control leak.

2. Two findings, one root cause

Both failures name the legacy reporting environment. One is a network path and the other is a load balancer listener, and they would be raised against different control families. Handled separately they produce two remediation tickets. Read together they describe one segment that predates the managed configuration path and was never brought inside it, which is why the project tracks the two drift findings together and expects one remediation to close both.

Drift tends to collect this way, around whatever was built before the current tooling and kept alive for one consumer. A program that reports per indicator should also ask what the failing indicators share.

3. Observed behavior governs, and exceptions are written down first

When the two methods disagree, the engine sets the indicator to `false` and records a `drift_analysis` block: the declared configuration satisfies the indicator, the observed behavior does not, and the observed behavior is authoritative.

The tempting alternatives are to treat the disagreement as telemetry noise, to average the two results, or to mark the indicator "partially met." The configuration is a claim about the system. The traffic is something the system did.

Telemetry can be wrong, and the example shows how that is handled without weakening the rule. Of 11 risk signals in the period, 9 led to an automatic account

disable. The other 2 came from the authorized external scanner and were suppressed by a documented allowlist entry (`ALW-017`). An observation is set aside only when a record explains it, and the record should predate the observation. The evidence here does not carry the date `ALW-017` was created, which is a gap: an explanation written after the failure is a finding with a rationale attached.

4. A failing indicator is evidence the program works

A package showing 13 of 13 would be less convincing than this one. A validator that has never returned a failure has not shown that it can.

The Ongoing Certification Report carries the result as it stands: 84.62% of indicators true, vulnerability detection coverage at 98.6% with two new hosts awaiting scanner enrollment, and one high-severity finding past its response expectation with a deviation request open.

Limits

Observed evidence is stronger than declared evidence, and it is still incomplete.

Observation has blind spots of its own. Flow logs see only traffic that occurred in the window. An active TLS scan sees only the endpoints it was pointed at. `KSI-PIY-GIV` reconciles the resource inventory against cloud discovery, which is the right pattern, but nothing in the evidence yet reconciles the 41 scanned endpoints against that inventory. Until it does, "40 of 41 compliant" is a statement about the scanner's target list.

Two of the thirteen pairings are not independent. Judging independence by what each method read, the engine reports `KSI-CMT-LMC` and `KSI-CMT-RMV` as not independently corroborated. Both of the change-logging methods read the same change-log record, and both of the deployment methods read the same pipeline record. Those two indicators pass on two views of one source. They carry less assurance than the other eleven and are reported that way.

The independence check is only as fine as the source list. A fixture stands in for one upstream system. If a real system served both configuration and telemetry, a check at this grain would treat them as one source, which is the cautious error. The check also works within an indicator and does not look across them: `KSI-IAM-AAM` and `KSI-IAM-ELP` both draw their telemetry from the same access-review job, so a fault in that job would affect two indicators at once.

Disagreement runs in both directions. The engine analyzes the case where configuration passes and behavior fails. The reverse, where behavior is clean and the declaration is missing, is flagged as drift and not analyzed. A system that behaves correctly by accident will not keep doing so.

Evidence about evidence needs a second witness too. The integrity manifest hashes each evidence file as written to disk, so `sha256sum` on a published file reproduces its entry and `--verify` fails on any mismatch. The test suite alters one value in a copied evidence file, and removes another file, and the check rejects both. That detects changes after generation. It does not detect a faulty generator, because the same process writes the evidence and the hashes. Signing by a separate identity, or publishing the hashes somewhere the generator cannot rewrite, would apply this paper's argument to its own outputs.

The Security Decision Record is a single snapshot. FedRAMP rule `SDR-CSX-KMT` requires a Class C provider to include historical KSI metrics: a 30-day summary, a summary of up to a year, and all daily metric data including persistent-validation status, where available. The generated record holds one evaluation and none of that history, so it does not demonstrate that rule. A snapshot also cannot show that validation is persistent, and `FRC-CSX-VVK` asks for methods that "persistently verify and validate."

The example is a sample. Thirteen indicators across seven of the ten KSI families, evaluated against fixtures that stand in for cloud, identity, and SIEM APIs. The method generalizes and the numbers are illustrative.

Applying this outside FedRAMP

None of this depends on a federal program. The same test applies to a control in a SOC 2, ISO 27001, or internal assurance context:

1. **Name the declaration and the observation for each control.** If you cannot say what behavior would be visible when the control works, the control is not yet testable.
2. **Check that the two come from different sources.** Ask whether one could be wrong while the other is right, and check what each one read instead of what it is called.
3. **Write pass criteria as "zero unexplained."** State in advance which exceptions are acceptable and what record makes them so.

4. **Decide which witness wins before the first disagreement.** A rule set under pressure gets negotiated.
5. **Validate the observer's coverage.** Reconcile what the scanner or log source can see against the asset inventory.
6. **Group findings by asset before assigning remediation.**
7. **Report failures as found,** with an owner and a date.

The shift from narrative assessment to automated validation is usually sold on speed. The better reason is that a narrative, a configuration export, and a single automated check all ask the system to describe itself, and a second independent method is the first point at which that description can be contradicted. An assurance program built that way stops asking whether a control is in place and starts asking what it would see if the control were not, and whether anyone has looked.

Scope and accuracy

- Salgado Public Systems, CaseBoard, and all metrics, endpoints, and findings are fictional. Figures are taken from the evidence files as generated for the 2026-08-10 evaluation.
- KSI identifiers and rule references (`FRC-CSX-VVK`, `SDR-CSO-FRR`, `CCM-OCR-AVL`, `FRC-CSO-JSN`) follow the FedRAMP Consolidated Rules for 2026 as implemented in Project 3. `KSI-CMT-RMV` quotes the official statement there; the other KSI statements are close paraphrases. I checked the text of `FRC-CSX-VVK` and `SDR-CSX-KMT` against fedramp.gov on 2026-10-13; `SDR-CSX-KMT` was last amended on 2026-10-08. The 2026 rules are revised often, so check any reference against the current text at fedramp.gov/2026.
- The JSON evidence is modeled on the rule requirements. It is a demonstration artifact: it is not validated against FedRAMP's published schemas and it omits the historical metrics `SDR-CSX-KMT` requires for Class C.
- This paper demonstrates methodology. It is not legal, regulatory, audit, or certification advice.

Source material

Artifact	Used for
`ksi_validator.py`	The validation engine, method pairings, and pass logic

Artifact	Used for
`test_validator.py`	Regression tests for runs A, B, and C and for the integrity check
`security-decision-record.json`	Per-indicator results, metrics, and drift analysis
`ongoing-certification-report.json`	Reported posture and open items
`evidence-integrity-manifest.json`	File hashes
KSI validation register (PDF)	Human-readable rendering of the results
Rev5 to 20x migration assessment (PDF)	Transition context
Tailspin Civic Rev5 package	The narrative-based model this is contrasted with